

PROBLEMA DO CAIXEIRO VIAJANTE MULTIMODAL

André Cardozo do Nascimento

Academia Militar das Agulhas Negras

Gabriel de Souza Santos

Academia Militar das Agulhas Negras

Henrique Chagas Aló

Academia Militar das Agulhas Negras

João Arthur Lima Cavalcante

Academia Militar das Agulhas Negras

Lucas Gimenis Teixeira Cardoso

Academia Militar das Agulhas Negras

Matheus Vieira Resende Sardinha

Academia Militar das Agulhas Negras

Piter Antônio da Silva

Academia Militar das Agulhas Negras

RESUMO

Este trabalho tem como objetivo desenvolver um Sistema de Apoio à Decisão para resolver uma variação multimodal do Problema do Caixeiro Viajante. A metodologia baseou-se na formulação matemática de Miller-Tucker-Zemlin (MTZ) para a eliminação de subrotas e no algoritmo exato *Branch-and-Cut*, utilizando o *solver* CBC integrado à linguagem Python. Os resultados demonstraram a eficácia do modelo em contornar a explosão combinatória, gerando soluções ótimas em tempo real. O sistema provou ser aderente à realidade, evidenciando o *trade-off* logístico entre a minimização de custos financeiros (priorizando o modal ferroviário) e a minimização do tempo de trânsito (utilização exclusiva do modal aéreo), auxiliando na tomada de decisão estratégica.

PALAVRAS CHAVE. Pesquisa Operacional. Problema do Caixeiro Viajante. Branch-and-Cut.

ABSTRACT

This work aims to develop a Decision Support System to solve a multimodal variation of the Traveling Salesperson Problem. The methodology was based on the Miller-Tucker-Zemlin (MTZ) mathematical formulation for subtour elimination and the exact Branch-and-Cut algorithm, using the CBC solver integrated with Python. The results demonstrated the model's effectiveness in bypassing the combinatorial explosion, generating optimal solutions in real time. The system proved to be applicable to real-world scenarios, highlighting the logistical trade-off between minimizing financial costs (prioritizing the railway mode) and minimizing transit time (exclusive use of the aerial mode), thus aiding in strategic decision-making.

KEYWORDS. Operations Research. Traveling Salesperson Problem. Branch-and-Cut.

1. Introdução

O planejamento de roteiros de viagem que envolvem múltiplos destinos é um desafio logístico comum e de alta complexidade. Quando um viajante decide visitar um conjunto de cidades, ele se depara com o problema de definir a melhor ordem de visita para que todas as localidades sejam percorridas exatamente uma vez, retornando ao ponto de partida no final. Esse cenário descreve, na prática, uma variação do clássico Problema do Caixeiro Viajante.

A escolha deste problema justifica-se pela necessidade de equilibrar tempo e dinheiro em viagens. Planejamentos intuitivos costumam gerar rotas ineficientes, com custos elevados ou perda de tempo em deslocamentos. Assim, uma abordagem de otimização estruturada maximiza a experiência no destino ao reduzir o desgaste logístico.

O objetivo principal deste trabalho é desenvolver um modelo de Sistema de Apoio à Decisão capaz de traçar a rota contínua ideal por um conjunto selecionado de cidades. Como objetivo secundário, o sistema deve atuar de forma multimodal, ou seja, além de definir o trajeto, ele deve escolher automaticamente o meio de transporte mais adequado (avião ou trem) para cada trecho, permitindo ao usuário visualizar cenários distintos focados na economia financeira ou na rapidez do deslocamento.

Para que o problema seja resolvido, o ambiente foi estruturado a partir de parâmetros predefinidos e variáveis de decisão. Os parâmetros representam os dados conhecidos e fixos do problema. Neste escopo, eles incluem:

- O conjunto de cidades que obrigatoriamente devem ser visitadas.
- As distâncias geográficas entre essas cidades.
- Os modais de transporte disponíveis (aéreo e ferroviário), considerando as restrições geográficas.
- As estimativas de custo financeiro e de tempo de duração da viagem entre cada par de cidades, dependendo do meio de transporte.

Já as variáveis representam as decisões que o modelo precisará tomar para encontrar a solução. Elas definem:

- A sequência exata de deslocamento (se o viajante deve ou não ir de uma cidade específica para outra).
- Qual o meio de transporte selecionado para realizar esse trecho específico.
- A ordem cronológica de chegada em cada cidade, garantindo que o itinerário forme um ciclo único e contínuo, sem criar rotas isoladas no meio do caminho.

Cabe ressaltar que o desenvolvimento computacional (scripts em Python via biblioteca PuLP), a formulação matemática e a revisão deste documento foram realizados com o auxílio do modelo de linguagem generativa Gemini, utilizado como assistente de produtividade algorítmica.

2. Modelagem Matemática

O problema abordado pode ser formulado como uma extensão multimodal do Problema do Caixeiro Viajante. A modelagem baseia-se na teoria de grafos e na formulação clássica de restrição para a eliminação de subrotas proposta originalmente na literatura matemática (Miller et al., 1960).

Índices e Conjuntos:

- $i, j \in \{1, 2, 3, \dots, n\}$: Índices representativos dos nós (cidades) pertencentes à rede, onde n é o número total de localidades.
- $k \in \{1, 2\}$: Índices representativos dos modais de transporte disponíveis (sendo $k = 1$ para modal aéreo e $k = 2$ para modal ferroviário).

Parâmetros:

- C_{ijk} : Custo financeiro estimado para o deslocamento da cidade i para a cidade j através do modal k .
- T_{ijk} : Tempo total de trânsito da cidade i para a cidade j através do modal k .
- P_{ijk} : Matriz de penalidade genérica definida pelo tomador de decisão, onde

$$P_{ijk} \in \{C_{ijk}, T_{ijk}\}$$

Variáveis de Decisão:

- x_{ijk} : Variável binária que assume o valor 1 se o deslocamento da cidade i para a cidade j for realizado utilizando o modal k e assume o valor 0 caso contrário.
- u_i : Variável contínua auxiliar associada a cada cidade i , indicando sua posição ou ordem de visita no roteiro (utilizada estritamente para a restrição de quebra de sub-rotas).

Função Objetivo:

O modelo busca minimizar o impacto financeiro ou temporal do circuito completo, de acordo com o parâmetro selecionado P_{ijk} .

$$\min z = \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^2 P_{ijk} x_{ijk}$$

Restrições Submetidas:

- Garante que o viajante parta de cada cidade i exatamente uma vez, utilizando apenas um único modal k com destino a uma cidade j .

$$\sum_{i=1, j \neq i}^n \sum_{k=1}^2 x_{ijk} = 1 \quad \forall i \in \{1, 2, \dots, n\}$$

- Garante que o viajante chegue a cada cidade j exatamente uma vez, proveniente de uma única cidade i através de um modal k .

$$\sum_{j=1, i \neq j}^n \sum_{k=1}^2 x_{ijk} = 1 \quad \forall i \in \{1, 2, \dots, n\}$$

- Impede a formação de múltiplos circuitos isolados e disjuntos (sub-rotas), assegurando que todas as cidades estejam conectadas em um único ciclo contínuo (formulação de MTZ).

$$u_i - u_j + n \sum_{k=1}^2 x_{ijk} \leq n - 1 \quad \forall i, j \in \{2, 3, \dots, n\}, i \neq j$$

Domínio das Variáveis:

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in \{1, 2, 3, \dots, n\}, \quad \forall k \in \{1, 2\} \quad u_i \geq 0 \quad \forall i \in \{2, 3, \dots, n\}$$

3. Metodologia

Para solucionar o modelo matemático proposto e contornar a explosão combinatória característica de problemas NP-Difíceis, a abordagem metodológica baseou-se na implementação de um Sistema de Apoio à Decisão (SAD) codificado integralmente na linguagem Python.

A arquitetura do sistema foi segmentada em três camadas principais: Processamento de Dados, Otimização Algorítmica e Interface de Usuário. Todos os códigos-fonte, bancos de dados (arquivos .csv) e *scripts* gerados nesta resolução encontram-se em anexo a este relatório para conferência.

3.1. Processamento Geográfico e Regras de Negócio

A estruturação da matriz de custos operacionais e tempos de deslocamento foi viabilizada através das bibliotecas *pandas* e *numpy*. Para o cálculo real das distâncias entre os nós da rede, o sistema emprega a Fórmula de Haversine, que considera a esfericidade da Terra para calcular a distância geodésica entre dois pontos a partir de suas latitudes e longitudes. A partir dessa distância base, os custos financeiros e tempos de duração dos modais aéreo e ferroviário foram definidos por funções lineares determinísticas, compostas por uma taxa fixa de embarque somada a um custo/tempo marginal por quilômetro percorrido.

Fórmula de Haversine

$$d = 2r \arcsin(\sqrt{\sin^2(\frac{\Delta\phi}{2}) + \cos\phi_1 \cos\phi_2 \sin^2(\frac{\Delta\lambda}{2})}), \text{ onde:}$$

d : distância final em quilômetros;

r : é o raio da terra (6371 km);

ϕ_1 e ϕ_2 : latitudes da cidade de origem e destino (convertidas para radianos);

$\Delta\phi$ e $\Delta\lambda$: diferenças absolutas entre as latitudes e longitudes de ambos os pontos.

Custo financeiro (R\$):

$$C_{aéreo} = 200 + 0,8d$$

$$C_{trem} = 50 + 0,4d$$

Tempo total (horas):

$$T_{aéreo} = 3 + \frac{d}{800}$$

$$T_{trem} = 1 + \frac{d}{250}$$

Adicionalmente, implementou-se uma sub-rotina de barreira geográfica baseada em conjuntos. Essa lógica impede que o algoritmo conecte ilhas isoladas (como o Japão ou o Reino Unido) a blocos continentais utilizando o modal ferroviário, penalizando essas combinações e forçando o modelo a adotar o modal aéreo nesses trechos específicos.

3.2. Motor de Otimização (Solver Exato)

A tradução das equações matemáticas para o ambiente computacional foi realizada através da biblioteca PuLP, um framework de modelagem em linguagem Python amplamente validado na literatura acadêmica (Mitchell et al., 2011). A fim de evitar a dependência de métodos heurísticos, a implementação baseou-se no código aberto COIN-OR (2026). Nesse sentido, a escolha recaiu sobre o solver exato CBC (Coin-or branch and cut), que conta com o suporte e manutenção do projeto COIN-OR (Lougee-Heimer, 2003).

Para garantir a integridade logística do circuito, implementou-se a formulação de Miller-Tucker-Zemlin (MTZ), que introduz variáveis contínuas auxiliares de ordem de visita para impedir a formação de sub-rotas disjuntas. Ao estabelecer que o índice cronológico de um destino seja estritamente superior ao de sua origem imediata, a restrição MTZ assegura a conectividade em um ciclo Hamiltoniano único, mantendo o modelo linearmente escalável em termos de número de restrições. Adicionalmente, o sistema executa ciclos iterativos distintos para processar a minimização financeira e temporal, permitindo uma análise rigorosa de *trade-offs* e a caracterização da Fronteira de Pareto para o problema multimodal proposto.

```
for i in range(1, n_cidades):
    for j in range(1, n_cidades):
        if i != j: prob += u[i] - u[j] + n_cidades * pulp.lpSum(x[i, j, k] for k in range(n_modais)) <= n_cidades - 1
prob.solve(pulp.PULP_CBC_CMD(msg=False))
```

Figura 1: Implementação iterativa do núcleo de restrições MTZ no PuLP.

3.3. Interface Gráfica e Hospedagem em Nuvem

A camada de interação com o tomador de decisão foi construída com o *framework* Streamlit. A renderização do traçado logístico é devolvida ao usuário através de projeções ortográficas 3D interativas (*globos*), viabilizadas pela biblioteca Plotly, permitindo a exploração visual dos roteiros e o detalhamento dos modais por meio de painéis flutuantes dinâmicos.

Visando garantir a portabilidade e a usabilidade do protótipo, a aplicação foi submetida a um processo de integração contínua (CI/CD) via repositório no *GitHub* e hospedada em arquitetura *serverless* através do *Streamlit Community Cloud*. Esta infraestrutura em nuvem permite que o modelo de Pesquisa Operacional seja acessado, testado e validado em tempo real por qualquer dispositivo conectado à internet.

4. Conclusão

O presente trabalho atingiu com êxito os objetivos propostos ao modelar, estruturar e resolver um problema logístico complexo de roteamento global, caracterizado como uma variação multimodal do Problema do Caixeiro Viajante (TSP). A aplicação da restrição de Miller-Tucker-Zemlin (MTZ) garantiu a eliminação de qualquer sub-rotas, validando a eficácia do modelo perante os desafios clássicos do Problema do Caixeiro Viajante descritos na literatura (Dantzig et al., 1954). O uso de solvers exatos garante que a solução apresentada não é apenas uma estimativa boa, mas sim a otimalidade matemática global provada para os parâmetros fornecidos (Hillier e Lieberman, 2010).

A análise dos resultados demonstra forte coerência matemática e estrita pertinência com o mundo real. O Sistema de Apoio à Decisão foi capaz de gerar cenários práticos de *trade-off* logístico: ao solicitar a minimização de custos, o modelo otimizou o orçamento utilizando massivamente a malha ferroviária; ao solicitar a minimização do tempo, o sistema priorizou rotas aéreas de alta velocidade. Além disso, o respeito às barreiras geográficas — como a proibição algorítmica de trens conectando ilhas ao continente — atesta que o modelo desenvolvido é maduro e aplicável a situações fáticas da vida real.

Como sugestões de evolução e melhoria para desenvolvimentos futuros, propõe-se: substituir as funções de custo determinísticas por um consumo dinâmico de APIs do mercado aéreo, capturando a volatilidade real e diária do preço das passagens. Além disso, incorporar restrições de horários de chegada e partida e um API que consiga adquirir a coordenada de qualquer local do mundo em tempo real.

Referências

COIN-OR. **Cbc (Coin-or branch and cut)**. Repositório GitHub, 2026. Disponível em: <https://github.com/coin-or/Cbc>. Acesso em: 29 jun. 2026.

DANTZIG, G. B.; FULKERSON, R.; JOHNSON, S. M. Solution of a large-scale traveling-salesman problem. **Journal of the Operations Research Society of America**, v. 2, n. 4, p. 393-410, 1954.

HILLIER, F. S.; LIEBERMAN, G. J. **Introdução à Pesquisa Operacional**. 8. ed. Porto Alegre: AMGH Editora, 2010.

LOUGEE-HEIMER, R. The Common Optimization Interface for Operations Research: Promoting open-source software in the operations research community. **IBM Journal of Research and Development**, v. 47, n. 1, p. 57-66, 2003.

MILLER, C. E.; TUCKER, A. W.; ZEMLIN, R. A. Integer programming formulation of traveling salesman problems. **Journal of the ACM**, v. 7, n. 4, p. 326-329, 1960.

MITCHELL, S.; O'SULLIVAN, M.; DUNNING, I. **PuLP: a linear programming toolkit for python**. The University of Auckland, Auckland, New Zealand, 2011.